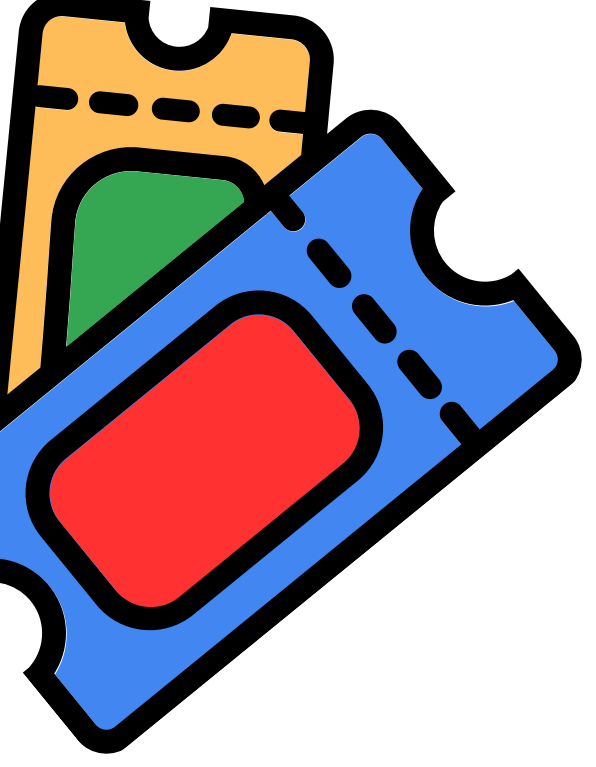
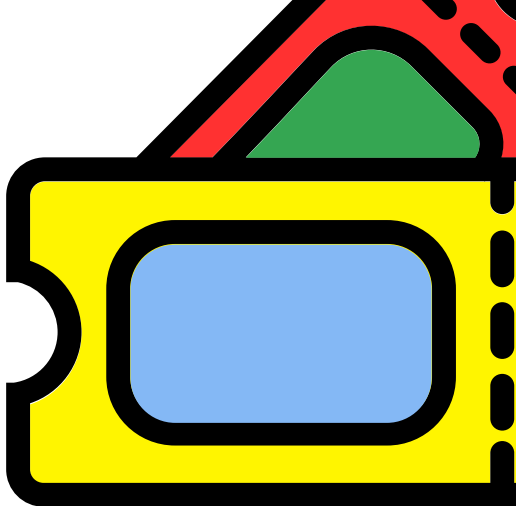


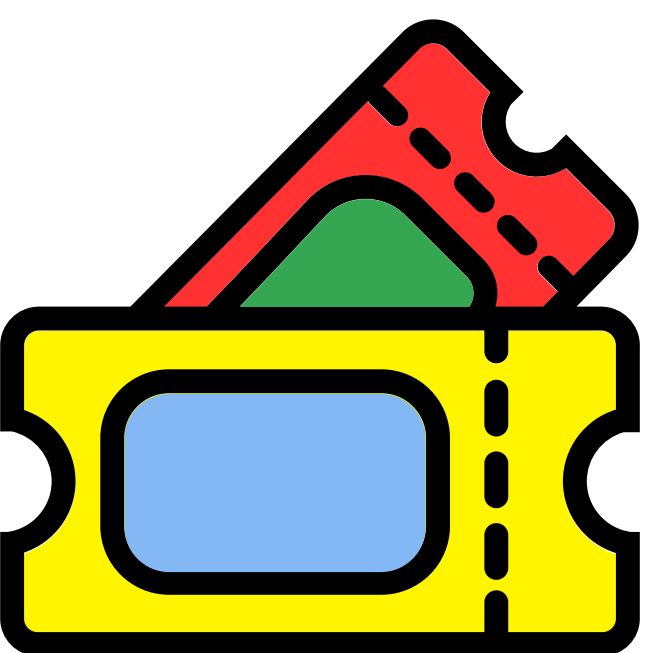


Google Developer Groups
IIT Mandi



RACKHACK

THEMES



01

BELUGA



Problem Explanation:

Malware poses a significant threat to organizations and individuals by compromising security, privacy, and data integrity. Traditional antivirus solutions can be slow to update their databases and are often inaccessible to novice users who just want a quick check on a suspicious file.

Your task is to build a web application that performs static analysis on uploaded files (e.g., `.exe`, `.docx`, `.pdf`), flags malicious indicators, and provides a clear verdict (“Malicious” or “Clean”). This system must be easy to use, offer concise results, and emphasize static analysis (no sandboxing or dynamic execution).

Resources for Beginners:

- **Basic Malware Analysis:** Malware Unicorn's
- **Static Analysis Tools:**
 - YARA: Pattern matching
 - Pefile: Windows PE analysis
- **Web Development Basics** (if unfamiliar):
 - MDN Web Docs
 - W3Schools

Evaluation Metrics:

A GitHub repository link is expected as your submission, containing a README file with clear instructions on how to run your code.
(mandatory)

1. **Detection Accuracy:** Ratio of correctly identified malicious/clean files.
2. **False Positives & False Negatives:** How often the tool misclassifies the files.
3. **User Experience & Interface:** Intuitive file upload, clear results, and minimal friction.
4. **Security & Reliability:** Robustness against malicious file uploads, input validation.
5. **Scalability & Performance:** How quickly the system can process multiple concurrent uploads.

Examples :

- **Use Case:** A user suspects a recently downloaded `.exe` might contain macros or malicious scripts. They upload the file, and the scanner quickly checks for suspicious strings, embedded macros, or unusual file structures.
- **Verdict:**
 - Clean: “No malicious indicators found.”
 - Malicious: “High entropy and suspicious macro code detected.”



What We Expect:

- **Front-End:** React, Angular, Vue, or a simple HTML/CSS/JS setup for file upload and result display.
- **Back-End:** Node.js, Python (Flask/Django/FastAPI), or Go for handling upload, scanning, and response.
- **File Analysis:** Python scripts for static analysis (using YARA, pefile, etc.).
- **Database:** For storing scan logs and reference signatures (optional).

Additional Suggestions:

- **Bonus:** Implement a “risk factor” report showing which parts of the file triggered malicious flags.
- **Security First:** Validate file uploads, sanitize file names, and protect against large malicious uploads (e.g., by limiting file size).

For any queries, contact - Mohit (+91 8210368145), Piyush Panpaliya(+91 9370098594)

02

EVENT TICKETING SYSTEM

Problem Statement: Blockchain-Based Event Ticketing System

Problem Explanation:

Event ticket fraud, scalping, and opaque resale markets cause financial losses and frustration for both organizers and attendees. Centralized ticketing platforms often fail to ensure authenticity and fair pricing.

You are tasked with creating a blockchain-powered ticketing solution (preferably a web application interacting with smart contracts deployed on the test net), where each ticket is minted as a unique NFT. This makes duplication impossible and provides verifiable ownership. Include a secondary marketplace with enforced price controls and royalties on every resale so that event organizers always receive a percentage, preventing exploitative markups and ensuring authenticity.

Resources for Beginners:

- **Basic Malware Analysis: Malware Unicorn's Static Analysis Tools:**
- **YARA: Pattern matching Pefile: Windows PE analysis**
- **Web Development Basics (if unfamiliar): MDN Web Docs W3Schools**

Evaluation Metrics:

A GitHub repository link is expected as your submission, containing a README file with clear instructions on how to run your code. (mandatory).

1. **Integrity & Authenticity:** Each ticket's on-chain metadata must be tamper-proof.
2. **Scalability:** The number of transactions the system can handle (important for large events).
3. **User Experience:** Simplicity of buying, selling, and transferring tickets even for non-crypto-savvy users.
4. **Fair Pricing:** Mechanisms to prevent scalping and excessive markups.
5. **Smart Contract Quality:** Security, clarity, and efficiency of the code.

Examples :

- **Concert Ticket Purchase:** Alice buys a ticket on the platform and holds the NFT in her wallet. On event day, the QR code linked to the NFT is scanned for entry.
- **Resale:** Bob wants to resell his ticket; the on-chain contract enforces a maximum resale price and routes a royalty to the event organizer automatically.



What We Expect:

- **Blockchain Layer:** Ethereum, Polygon, or other EVM-compatible chains.
- **Smart Contract Development:** Solidity (using frameworks like Hardhat or Truffle).
- **Frontend:** React or Vue with Web3 libraries (ethers.js/web3.js).
- **NFT Standards:** ERC-721 or ERC-1155 for ticket tokens.
- **Optional:** Payment gateway integration for fiat on-ramps (e.g., Stripe crypto).

Additional Suggestions:

- **Bonus Features:**
 - **Bidding System:** Auction-based approach for premium seats.
 - **Anti-Bot Measures:** Smart contract-based purchase limits.
 - **Dynamic Ticketing:** Update seat details or perks post-mint without reissuing.

For any queries, contact - James (+91 8849556496)



03 TIME CAPSULE 2.0



Problem Explanation:

A “digital time capsule” provides a unique way to preserve memories, cultural heritage, or personal milestones over time, benefiting communities, families, or organizations. Many existing solutions are fragmented (e.g., social media, photo albums), lacking a central, secure, and engaging platform for collaborative memory preservation.

Your goal is to build a web platform for users to create, store, and share digital “capsules” of text, images, and videos. These capsules can be locked until a future date, opened collectively for significant events, or used as a communal space to record shared experiences (e.g., disaster recovery stories or group journaling).

Resources for Beginners:

- **Front-End:**
 - React Or Vue.js
- **Back-End:**
 - Node.js / Express
 - Django Or Ruby on Rails
- **Databases:**
 - MongoDB
 - PostgreSQL
- **Hosting:**
 - Heroku Or AWS Or Netlify

Evaluation Metrics:

A GitHub repository link is expected as your submission, containing a README file with clear instructions on how to run your code.
(mandatory)

1. **Impact & Relevance:** How effectively the platform addresses real-world preservation needs (personal, cultural, or communal).
2. **Innovation & Creativity:** Novel ways to curate or contextualize memories, e.g., AI-based recommendations.
3. **User Experience & Design:** Intuitive navigation, appealing UI, easy upload.
4. **Security & Privacy:** Ensuring data confidentiality, user authentication, encryption.
5. **Scalability:** Handling large media files and potentially thousands of concurrent users.

Examples :

- **Community Capsule:** A local community collects stories, photos, and videos about a cultural festival to open on its 10th anniversary.
- **Personal Wellness:** A user creates a gratitude journal capsule locked for six months, revisiting entries later to track personal growth.



What We Expect:

- **Core Web Tech:** HTML, CSS, JavaScript.
- **Framework:** React/Angular/Vue for front-end + Node/Django/Flask for back-end.
- **Media Handling:** S3 or Firebase Storage to handle images and videos.
- **Authentication:** OAuth2 or JWT-based user login.

Additional Suggestions:

- **AI/ML Integration:** Sentiment analysis or photo tagging to organize capsules.
- **Gamification:** Unlock achievements for consistent journaling or communal interactions.
- **Optional Blockchain:** Store a hash of each “capsule” for immutability and authenticity.

**For any queries, contact - Mehul Sharma(+91 7580021658),
Vinamra Garg(+91 9350555053)**

04 IIT MANDI CAMPUS MARKETPLACE



Problem Explanation:

Within a university campus, students, faculty, and staff often need a centralized platform to buy, sell, or exchange items (like textbooks, electronics, or services). Relying on social media or word of mouth is inefficient and lacks structure or security features.

Create a mobile application tailored to IIT Mandi (or any similar campus) that consolidates listings into categories, includes secure communication between buyers and sellers, and fosters a trusted community marketplace.

Resources for Beginners:

- **Mobile Development:**
 - Android: Android Developers
 - iOS: Apple Developer
 - Cross-platform: React Native Or Flutter
- **Backend APIs:**
 - Firebase: Quick setup for authentication & database
 - Express.js + MongoDB or any RESTful approach
- **UI/UX Design:**
 - Figma Or Adobe XD

Evaluation Metrics:

A GitHub repository link is expected as your submission, containing a README file with clear instructions on how to run your code.
(mandatory)

1. **Functionality & Completeness:** Implementation of core marketplace features (listings, search, messaging).
2. **User Experience & UI:** Intuitive design for quick listing and browsing.
3. **Security & Trust:** Ability to rate/review sellers, handle potential scams or misuse.
4. **Performance:** Smooth handling of images, real-time notifications, minimal loading times.
5. **Innovation & Additional Features:** Payment gateway integration, location-based listings, etc.

Examples :

- **Selling Used Electronics:** A student posts a used laptop with images, price, and condition details. Interested buyers can message or comment.
- **Book Exchange:** A faculty member lists old textbooks at a discounted price, or offers an exchange for specific titles.



What We Expect:

- **Front-End:** Flutter/React Native or Native Android/iOS.
- **Back-End:** Node.js (Express), Python (Flask/FastAPI), or Firebase.
- **Database:** Firestore, MongoDB, or PostgreSQL.
- **Authentication:** Email-password or institutional single sign-on (if applicable).

Additional Suggestions:

- **Notification System:** Real-time push notifications for new messages or offers.
- **In-app Chat:** Dedicated messaging to protect privacy (avoid sharing phone numbers).
- **Rating & Review System:** Build trust with community-driven feedback.

For any queries, contact - Aman Sikarwar (+91 97199 62248)

05 INTELLIGENT TRAFFIC PERCEPTION



Problem Explanation:

With growing urban populations and congested roads, real-time traffic monitoring is essential for road safety and efficiency. Many existing systems either detect a single object category (e.g., only vehicles) or rely on outdated sensor technology.

Your task is to develop a deep learning model that processes traffic video in real time, identifying multiple classes (vehicles, pedestrians, traffic signs, lanes, etc.), outputting structured data to help city planners, law enforcement, or autonomous vehicles.

Resources for Beginners:

- **Dataset:**
 - BDD100K: Large-scale annotated driving videos
- **Model Architectures:**
 - YOLOv8
 - Faster R-CNN
 - Detectron2
- **Deep Learning Frameworks:**
 - PyTorch
 - TensorFlow
- **Tutorials:**
 - PyTorch Object Detection

Evaluation Metrics:

A GitHub repository link is expected as your submission, containing a README file with clear instructions on how to run your code and a google drive link containing the weights for your Deep Learning Model (Mandatory). Deployment of your code will earn you bonus points.

1. MAP (Mean Average Precision): Accuracy for detecting multiple object classes.
2. IoU (Intersection over Union): Overlap between predicted and ground truth bounding boxes.
3. Inference Speed (FPS): Real-time performance (frames per second).
4. Robustness: Performance under diverse lighting, weather, and traffic conditions.
5. Innovation: Implementation of novel architectures (e.g., transformers, attention mechanisms).

Examples :

- **Traffic Sign Detection:** Detecting stop signs, speed limits, and yield signs in suburban or highway footage.
- **Vehicle Tracking:** Identifying car models, counting trucks vs. motorcycles, or tracking specific lanes.
- **Pedestrian Alerts:** Locating and tracking people in crosswalks for accident prevention.



What We Expect:

- **Model Training:** PyTorch or TensorFlow
- **Data Pipeline:** Python scripts for frame extraction, data augmentation, etc.
- **Inference/Deployment:**
 - Python/Flask/FastAPI or a C++/TensorRT pipeline for real-time performance
 - Optional: ONNX or TensorRT optimization
- **Visualization:** OpenCV or a web interface to display bounding boxes on live video .

**For any queries, contact - Vinamra Garg (+91 9350555053),
Luv Sharma(+91 79822 99553)**



06 AUTONOMOUS APPLICATION BUILDER



Problem Explanation:

Imagine a system where a user simply describes their idea—for example, "Create an e-commerce site for handmade crafts"—and a network of specialized AI agents takes over. Using the tools provided by [agent.ai\(https://agent.ai/builders\)](https://agent.ai/builders), your challenge is to develop a collaborative multi-agent system that automatically designs, builds, and deploys a web application or website based solely on the prompt.

Key Agent Roles:

- **Designer Agent:** Responsible for generating visual layouts, UI/UX design components, and branding elements.
- **Developer Agent:** Writes the backend and frontend code, selects frameworks, and ensures that all components work seamlessly together.
- **Project Manager Agent:** Oversees the project, coordinates the communication between agents, and manages deployment pipelines (e.g., setting up hosting, integrating CI/CD).

The agents must communicate and collaborate effectively to produce a final deliverable—a deployed, fully functional application that goes beyond merely generating code.

Evaluation Metrics:

A GitHub repository link is expected as your submission, containing a README file with clear instructions on how to run your code. (mandatory) Deployment of your code will earn you bonus points.

1. **End-to-End Functionality:** The final product must be a deployed application that meets the user's requirements.
2. **Design Quality & Usability:** The UI/UX should be visually appealing and intuitive.
3. **Code Quality & Integration:** Agents must generate maintainable, scalable code with minimal errors and seamless integration between components.
4. **Agent Collaboration Efficiency:** How effectively do the agents coordinate tasks, share information, and resolve conflicts?
5. **Deployment Robustness:** The solution should include automated testing, error handling, and a smooth deployment process (e.g., using CI/CD pipelines).

Examples :

- **Scenario 1:**

- Prompt: “Build a restaurant reservation web app with online ordering.”
- Outcome:
 - The Designer Agent creates an attractive, responsive UI.
 - The Developer Agent sets up the database, APIs, and front-end functionality (e.g., reservation system, menu browsing).
 - The Project Manager Agent orchestrates testing and deploys the app on a cloud service with automated scaling.

• **Scenario 2:**

- **Prompt:** “Create a personal blog with integrated social media feeds and dynamic content.”
- **Outcome:**
 - The agents produce a modern, blog-ready design; integrate content management and social media APIs; and deploy the website with continuous updates.

What We Expect:

- **Front-End:** React, Vue.js, or Angular for responsive user interfaces.
- **Back-End:** Node.js, Python (Flask, Django), or similar for API development.
- **Database:** PostgreSQL, MongoDB, or Firebase for data management.
- **Agent Framework:** agent.ai’s platform for creating and managing AI agents.
- **Deployment:** Docker, Kubernetes, or serverless platforms (e.g., AWS Lambda, Vercel) along with CI/CD pipelines (GitHub Actions, Jenkins).
- **Communication Protocols:** RESTful APIs or WebSocket for inter-agent communication.

For any queries, contact - Mahir Jain (+91 6375309975)

FOLLOW US:



Instagram



Linkedin

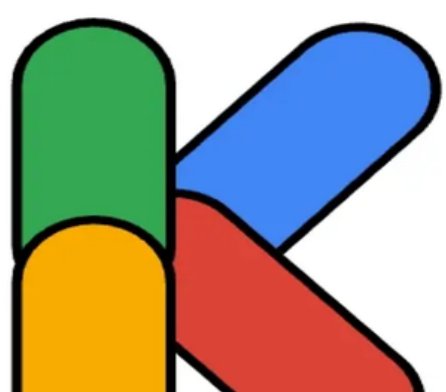


X



Website

RACKHACK



RACKHACK